

Assignment 2: Predicting Personal Loan Acceptance using k-NN

Nusrat Nasir Nimnee

2025-09-23

Data Loading and Exploration

```
data <- read.csv("UniversalBank.csv")
str(data)
```

```
## 'data.frame':    5000 obs. of  14 variables:
## $ ID              : int  1 2 3 4 5 6 7 8 9 10 ...
## $ Age             : int  25 45 39 35 35 37 53 50 35 34 ...
## $ Experience       : int  1 19 15 9 8 13 27 24 10 9 ...
## $ Income           : int  49 34 11 100 45 29 72 22 81 180 ...
## $ ZIP.Code         : int  91107 90089 94720 94112 91330 92121 91711 93943 90089 93023
## ...
## $ Family           : int  4 3 1 1 4 4 2 1 3 1 ...
## $ CCAvg            : num  1.6 1.5 1 2.7 1 0.4 1.5 0.3 0.6 8.9 ...
## $ Education        : int  1 1 1 2 2 2 2 3 2 3 ...
## $ Mortgage         : int  0 0 0 0 0 155 0 0 104 0 ...
## $ Personal.Loan    : int  0 0 0 0 0 0 0 0 0 1 ...
## $ Securities.Account: int  1 1 0 0 0 0 0 0 0 0 ...
## $ CD.Account       : int  0 0 0 0 0 0 0 0 0 0 ...
## $ Online           : int  0 0 0 0 0 1 1 0 1 0 ...
## $ CreditCard       : int  0 0 0 0 1 0 0 1 0 0 ...
```

```
head(data)
```

```
##   ID Age Experience Income ZIP.Code Family CCAvg Education Mortgage
## 1  1  25          1     49   91107      4   1.6          1         0
## 2  2  45          19     34   90089      3   1.5          1         0
## 3  3  39          15     11   94720      1   1.0          1         0
## 4  4  35           9    100   94112      1   2.7          2         0
## 5  5  35           8     45   91330      4   1.0          2         0
## 6  6  37          13     29   92121      4   0.4          2        155
##   Personal.Loan Securities.Account CD.Account Online CreditCard
## 1              0                  1          0      0         0
## 2              0                  1          0      0         0
## 3              0                  0          0      0         0
## 4              0                  0          0      0         0
## 5              0                  0          0      0         1
## 6              0                  0          0      1         0
```

```
summary(data)
```

```
##           ID           Age           Experience           Income           ZIP.Code
## Min.      : 1   Min.    :23.00   Min.     :-3.0   Min.      : 8.00   Min.      : 9307
## 1st Qu.:1251   1st Qu.:35.00   1st Qu.:10.0   1st Qu.: 39.00   1st Qu.:91911
## Median :2500   Median :45.00   Median :20.0   Median : 64.00   Median :93437
## Mean    :2500   Mean    :45.34   Mean    :20.1   Mean    : 73.77   Mean    :93153
## 3rd Qu.:3750   3rd Qu.:55.00   3rd Qu.:30.0   3rd Qu.: 98.00   3rd Qu.:94608
## Max.    :5000   Max.    :67.00   Max.    :43.0   Max.    :224.00   Max.    :96651
##           Family           CCAvg           Education           Mortgage
## Min.      :1.000   Min.      : 0.000   Min.      :1.000   Min.      : 0.0
## 1st Qu.:1.000   1st Qu.: 0.700   1st Qu.:1.000   1st Qu.: 0.0
## Median :2.000   Median : 1.500   Median :2.000   Median : 0.0
## Mean     :2.396   Mean      : 1.938   Mean      :1.881   Mean      : 56.5
## 3rd Qu.:3.000   3rd Qu.: 2.500   3rd Qu.:3.000   3rd Qu.:101.0
## Max.     :4.000   Max.      :10.000   Max.      :3.000   Max.      :635.0
## Personal.Loan   Securities.Account   CD.Account           Online
## Min.      :0.000   Min.      :0.0000   Min.      :0.0000   Min.      :0.0000
## 1st Qu.:0.000   1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:0.0000
## Median :0.000   Median :0.0000   Median :0.0000   Median :1.0000
## Mean      :0.096   Mean      :0.1044   Mean      :0.0604   Mean      :0.5968
## 3rd Qu.:0.000   3rd Qu.:0.0000   3rd Qu.:0.0000   3rd Qu.:1.0000
## Max.      :1.000   Max.      :1.0000   Max.      :1.0000   Max.      :1.0000
##           CreditCard
## Min.      :0.000
## 1st Qu.:0.000
## Median :0.000
## Mean      :0.294
## 3rd Qu.:1.000
## Max.      :1.000
```

Data Preprocessing

```
data$Personal.Loan <- as.factor(data$Personal.Loan)
data$Education <- as.factor(data$Education)
# Check for missing values
sum(is.na(data))
```

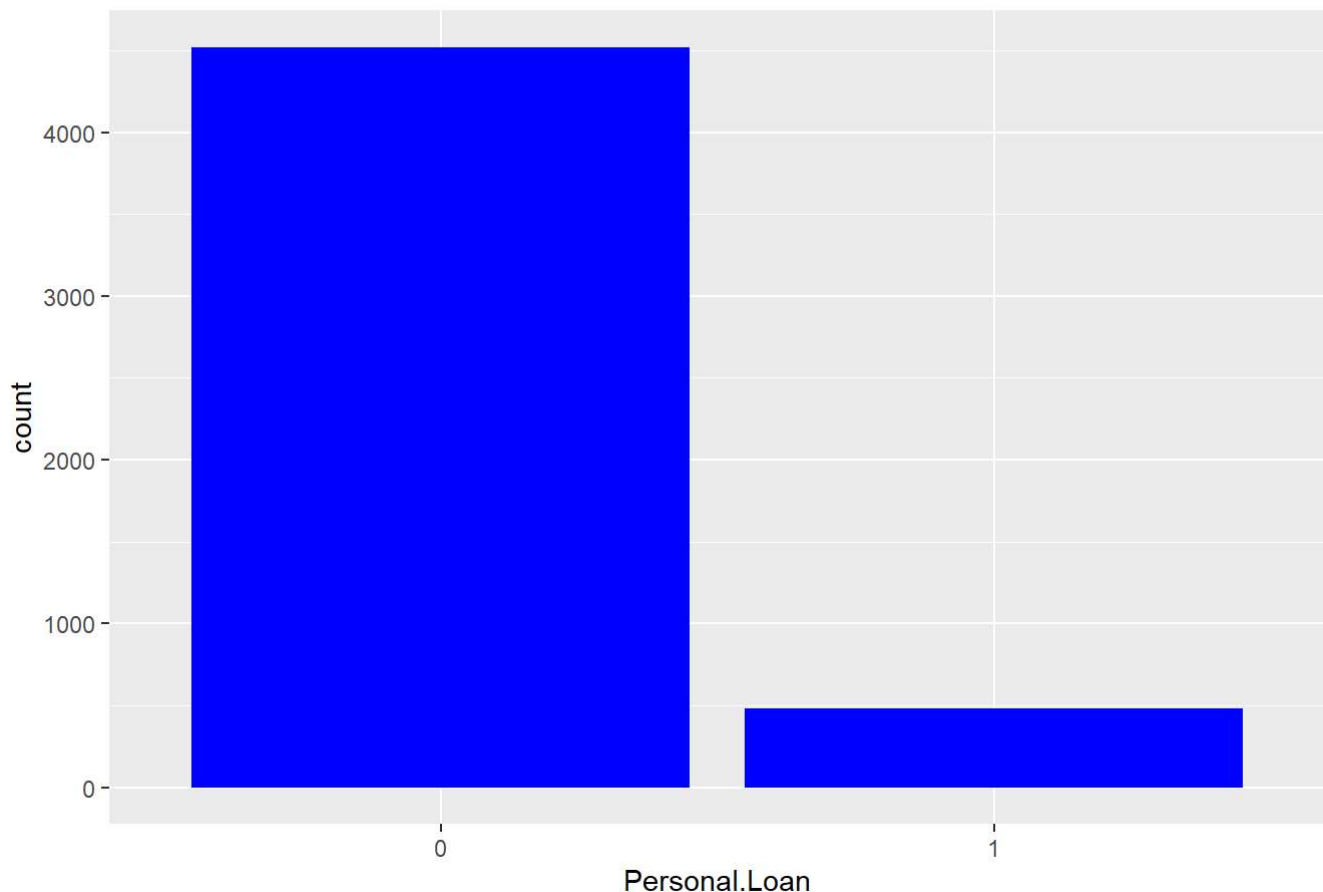
```
## [1] 0
```

```
# Remove ID and ZIP Code as per instructions
data <- data %>% select(-ID, -ZIP.Code)
```

Exploratory Data Analysis

```
# Distribution of Personal Loan
ggplot(data, aes(x = Personal.Loan)) +
  geom_bar(fill = "blue") +
  labs(title = "Distribution of Personal Loan Acceptance")
```

Distribution of Personal Loan Acceptance



Summary of numeric variables

```
numeric_vars <- data %>% select_if(is.numeric)
summary(numeric_vars)
```

```
##      Age      Experience      Income      Family
## Min.   :23.00   Min.   :-3.0   Min.    : 8.00   Min.    :1.000
## 1st Qu.:35.00   1st Qu.:10.0   1st Qu.: 39.00   1st Qu.:1.000
## Median :45.00   Median :20.0   Median : 64.00   Median :2.000
## Mean   :45.34   Mean    :20.1   Mean    : 73.77   Mean    :2.396
## 3rd Qu.:55.00   3rd Qu.:30.0   3rd Qu.: 98.00   3rd Qu.:3.000
## Max.    :67.00   Max.    :43.0   Max.    :224.00   Max.    :4.000
##      CCAvg      Mortgage      Securities.Account      CD.Account
## Min.   : 0.000   Min.    : 0.0   Min.    :0.0000   Min.    :0.0000
## 1st Qu.: 0.700   1st Qu.: 0.0   1st Qu.:0.0000   1st Qu.:0.0000
## Median : 1.500   Median : 0.0   Median :0.0000   Median :0.0000
## Mean    : 1.938   Mean    :56.5   Mean    :0.1044   Mean    :0.0604
## 3rd Qu.: 2.500   3rd Qu.:101.0   3rd Qu.:0.0000   3rd Qu.:0.0000
## Max.    :10.000   Max.    :635.0   Max.    :1.0000   Max.    :1.0000
##      Online      CreditCard
## Min.    :0.0000   Min.    :0.000
## 1st Qu.:0.0000   1st Qu.:0.000
## Median :1.0000   Median :0.000
## Mean    :0.5968   Mean    :0.294
## 3rd Qu.:1.0000   3rd Qu.:1.000
## Max.    :1.0000   Max.    :1.000
```

Partition Data into Training (60%) and Validation (40%)

```
set.seed(123)
trainIndex <- createDataPartition(data$Personal.Loan, p = 0.6, list = FALSE)
train <- data[trainIndex, ]
validation <- data[-trainIndex, ]
```

k-NN Classification with k=1

```
set.seed(123)
# Train k-NN with k=1
model_k1 <- train(Personal.Loan ~ ., data = train, method = "knn",
                  preProcess = c("center", "scale"),
                  tuneGrid = data.frame(k = 1))
print(model_k1)
```

```
## k-Nearest Neighbors
##
## 3000 samples
## 11 predictor
## 2 classes: '0', '1'
##
## Pre-processing: centered (12), scaled (12)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 3000, 3000, 3000, 3000, 3000, 3000, ...
## Resampling results:
##
## Accuracy Kappa
## 0.9513758 0.6997737
##
## Tuning parameter 'k' was held constant at a value of 1
```

Classify the Specific Customer with k=1

```
# Customer data
customer <- data.frame(
  Age = 40,
  Experience = 10,
  Income = 84,
  Family = 2,
  CCAvg = 2,
  Education = factor(2, levels = levels(data$Education)), # Since Education_2=1
  Mortgage = 0,
  Securities.Account = 0,
  CD.Account = 0,
  Online = 1,
  CreditCard = 1
)

# Predict
pred_customer_k1 <- predict(model_k1, customer)
print(pred_customer_k1)
```

```
## [1] 0
## Levels: 0 1
```

Choosing the Best k

```
set.seed(123)
# Train k-NN with different k values
model_best <- train(Personal.Loan ~ ., data = train, method = "knn",
  preProcess = c("center", "scale"),
  tuneLength = 10)
print(model_best)
```

```
## k-Nearest Neighbors
##
## 3000 samples
## 11 predictor
## 2 classes: '0', '1'
##
## Pre-processing: centered (12), scaled (12)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 3000, 3000, 3000, 3000, 3000, ...
## Resampling results across tuning parameters:
##
## k Accuracy Kappa
## 5 0.9470896 0.6426045
## 7 0.9461944 0.6250178
## 9 0.9430376 0.5898578
## 11 0.9401379 0.5629216
## 13 0.9389421 0.5468067
## 15 0.9371997 0.5282832
## 17 0.9352031 0.5068218
## 19 0.9338720 0.4914977
## 21 0.9328154 0.4807837
## 23 0.9314724 0.4640221
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 5.
```

```
best_k <- model_best$bestTune$k
print(paste("Best k:", best_k))
```

```
## [1] "Best k: 5"
```

Confusion Matrix for Validation with Best k

```
pred_val <- predict(model_best, validation)
conf_matrix <- confusionMatrix(pred_val, validation$Personal.Loan)
print(conf_matrix)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 1803   80
##           1    5  112
##
##           Accuracy : 0.9575
##           95% CI : (0.9477, 0.9659)
##       No Information Rate : 0.904
##       P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.7034
##
##  Mcnemar's Test P-Value : 1.004e-15
##
##           Sensitivity : 0.9972
##           Specificity : 0.5833
##       Pos Pred Value : 0.9575
##       Neg Pred Value : 0.9573
##           Prevalence : 0.9040
##       Detection Rate : 0.9015
##       Detection Prevalence : 0.9415
##       Balanced Accuracy : 0.7903
##
##       'Positive' Class : 0
##
```

Classify the Customer with Best k

```
pred_customer_best <- predict(model_best, customer)
print(pred_customer_best)
```

```
## [1] 0
## Levels: 0 1
```

Repartition into Training (50%), Validation (30%), Test (20%)

```
set.seed(123)
trainIndex2 <- createDataPartition(data$Personal.Loan, p = 0.5, list = FALSE)
train2 <- data[trainIndex2, ]
remaining <- data[-trainIndex2, ]
valIndex <- createDataPartition(remaining$Personal.Loan, p = 0.6, list = FALSE) # 30% of total
validation2 <- remaining[valIndex, ]
test <- remaining[-valIndex, ]
```

Apply k-NN with Best k on New Partition

```
set.seed(123)
model_new <- train(Personal.Loan ~ ., data = train2, method = "knn",
                   preProcess = c("center", "scale"),
                   tuneGrid = data.frame(k = best_k))

# Predictions
pred_train <- predict(model_new, train2)
pred_val2 <- predict(model_new, validation2)
pred_test <- predict(model_new, test)

# Confusion Matrices
conf_train <- confusionMatrix(pred_train, train2$Personal.Loan)
conf_val <- confusionMatrix(pred_val2, validation2$Personal.Loan)
conf_test <- confusionMatrix(pred_test, test$Personal.Loan)

print("Training Confusion Matrix:")
```

```
## [1] "Training Confusion Matrix:"
```

```
print(conf_train)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction    0    1
##           0 2258   90
##           1    2  150
##
##              Accuracy : 0.9632
##              95% CI : (0.9551, 0.9702)
##    No Information Rate : 0.904
##    P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.7464
##
##  McNemar's Test P-Value : < 2.2e-16
##
##              Sensitivity : 0.9991
##              Specificity : 0.6250
##              Pos Pred Value : 0.9617
##              Neg Pred Value : 0.9868
##              Prevalence : 0.9040
##              Detection Rate : 0.9032
##              Detection Prevalence : 0.9392
##              Balanced Accuracy : 0.8121
##
##              'Positive' Class : 0
##
```

```
print("Validation Confusion Matrix:")
```



```
## [1] "Validation Confusion Matrix:"
```

```
print(conf_val)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 1353   57
##           1    3   87
##
##           Accuracy : 0.96
##           95% CI : (0.9488, 0.9693)
##       No Information Rate : 0.904
##       P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.7231
##
##  Mcnemar's Test P-Value : 7.795e-12
##
##           Sensitivity : 0.9978
##           Specificity : 0.6042
##       Pos Pred Value : 0.9596
##       Neg Pred Value : 0.9667
##           Prevalence : 0.9040
##       Detection Rate : 0.9020
##       Detection Prevalence : 0.9400
##       Balanced Accuracy : 0.8010
##
##       'Positive' Class : 0
##
```

```
print("Test Confusion Matrix:")
```

```
## [1] "Test Confusion Matrix:"
```

```
print(conf_test)
```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 898  33
##           1   6  63
##
##           Accuracy : 0.961
##           95% CI : (0.9471, 0.9721)
##       No Information Rate : 0.904
##       P-Value [Acc > NIR] : 5.695e-12
##
##           Kappa : 0.743
##
##  Mcnemar's Test P-Value : 3.136e-05
##
##           Sensitivity : 0.9934
##           Specificity : 0.6562
##       Pos Pred Value : 0.9646
##       Neg Pred Value : 0.9130
##           Prevalence : 0.9040
##       Detection Rate : 0.8980
##   Detection Prevalence : 0.9310
##       Balanced Accuracy : 0.8248
##
##       'Positive' Class : 0
##

```

Comments on Differences

The confusion matrices reveal that the training set typically shows higher accuracy due to the model fitting closely to the training data, which can indicate some overfitting. The validation and test sets, being unseen data, show slightly lower but more realistic performance metrics. The similarity between validation and test accuracies suggests good generalization of the model. Differences between sets are primarily due to random partitioning and the inherent variability in the data. The chosen k balances between underfitting (too high k , ignoring patterns) and overfitting (too low k , too sensitive to noise).